



Anil Seth

Modelling the Shortage of Change Using Netlogo

Last month we explored Netlogo, the multi-agent programming environment which simulates natural and social phenomena. In this article, the author skilfully uses an existing real-world situation, namely, the shortage of change due to demonetisation, to set up a Netlogo model.

We have all recently experienced the shortage of cash or change. Since my expectations of the cash availability situation post demonetisation did not turn out to be correct, I wanted to see if a simple model could be created in Netlogo, to model the shortage of change.

The model

The simple model I chose was as follows:

- Our world consists of shops, shoppers, coins and notes.
- Each note is equal to note-value coins.
- There are a fixed number of shops, N_{shops} , at fixed locations.
- Each shop starts with starting-change coins.
- There are a fixed number of shoppers, $N_{shoppers}$, who move a step on each tick.
- Each person starts with no coins but an infinite number of notes.
- If a person lands at a shop, (s)he makes a purchase of a random value between 0 and note-value excluding the end points.
- Each person pays with coins if (s)he has enough and does not hold onto them.
- Each shop returns the change if it has enough, even if the sale is for a small amount.
- If neither the shop nor the shopper has an adequate number of coins, the sale fails.
- You can vary the values of N_{shops} , $N_{shoppers}$, starting-change and note-value to study the rate of failed sales.

The code

The interface screen is shown in Figure 1. This includes the values you can set for the number of shops and the number of shoppers in this world. You also configure the amount of change each shop has and the number of coins that add up to the value of a note.

You need to define 'turtles' called shops and shoppers, and various variables for the model. The global variables keep track of the total value of sales, the

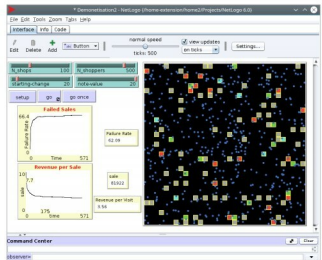


Figure 1: Interface for the demonetisation model

number of sales and the number of sales that failed because there was no change.

```
breed [shops shop]
breed [shoppers shopper]
shops-own [shop-coins]
shoppers-own [coins]
globals [sale no-change count-sales]
```

The set-up code sets up the shops at non-overlapping locations. Each shop is a square. The shoppers are created at random locations and shaped as dots.

```
to setup
  clear-all
  set sale 0
  setup-shops
  setup-shoppers
  reset-ticks
end
```

```
to setup-shops
```